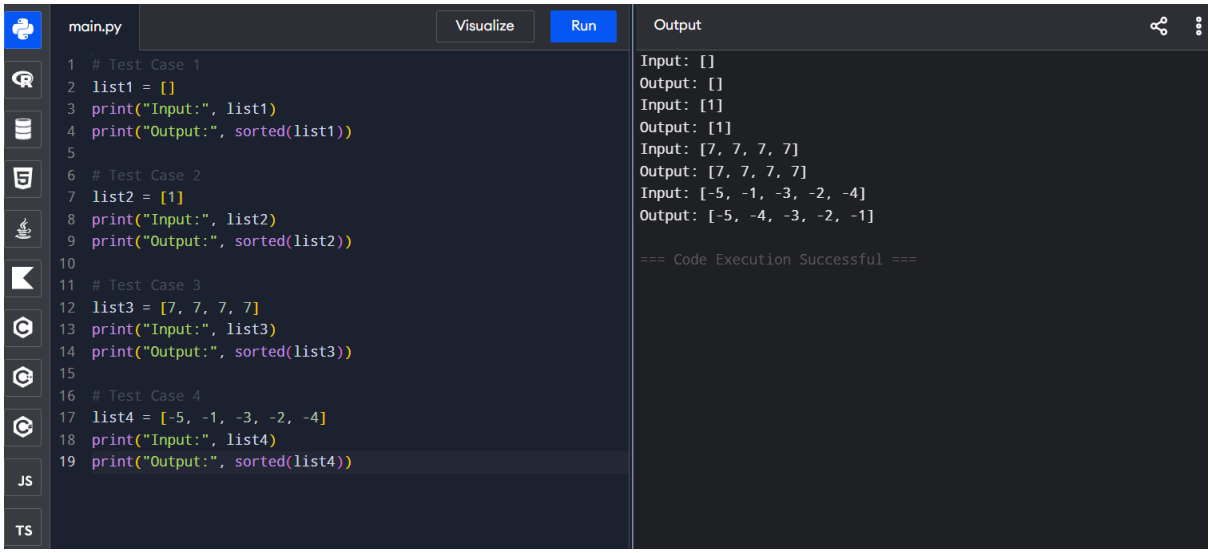
CHAPTER – 2

Name: Jeslet Jessica. T

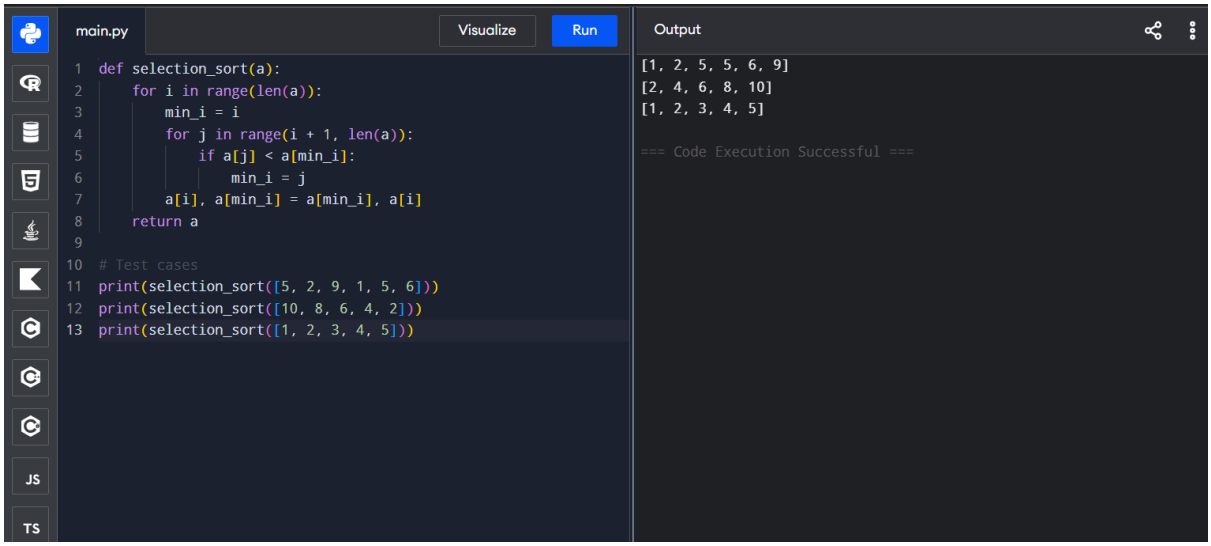
Regno: 192524135

Course: CSA0603 – DAA

EXPERIMENT – 1



EXPERIMENT – 2



EXPERIMENT – 3

main.py

```python
def bubble_sort(a):
    for i in range(len(a)):
        swapped = False
        for j in range(len(a)-1-i):
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                swapped = True
        if not swapped:
            break
    return a

print(bubble_sort([64, 25, 12, 22, 11]))
print(bubble_sort([29, 10, 14, 37, 13]))
print(bubble_sort([3, 5, 2, 1, 4]))
print(bubble_sort([1, 2, 3, 4, 5]))
print(bubble_sort([5, 4, 3, 2, 1]))
```

Output

```
[11, 12, 22, 25, 64]
[10, 13, 14, 29, 37]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

EXPERIMENT – 4

main.py

```python
def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1

        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1

        a[j + 1] = key

    return a

print(insertion_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3]))
print(insertion_sort([5, 5, 5, 5, 5]))
print(insertion_sort([2, 3, 1, 3, 2, 1, 1, 3]))
```

Output

```
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
[5, 5, 5, 5, 5]
[1, 1, 1, 2, 2, 3, 3, 3]

=== Code Execution Successful ===
```

EXPERIMENT – 5

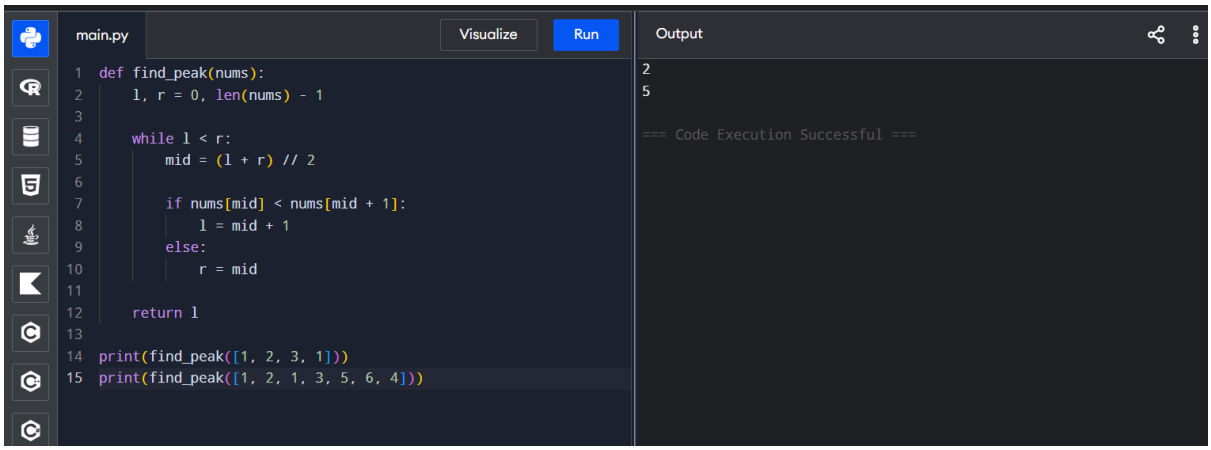
main.py

```python
def find_missing(arr, k):
    n = 1
    count = 0

    while count < k:
        if n not in arr:
            count += 1
        if count == k:
            return n
        n += 1

print(find_missing([2, 3, 4, 7, 11], 5))
print(find_missing([1, 2, 3, 4], 2))
```

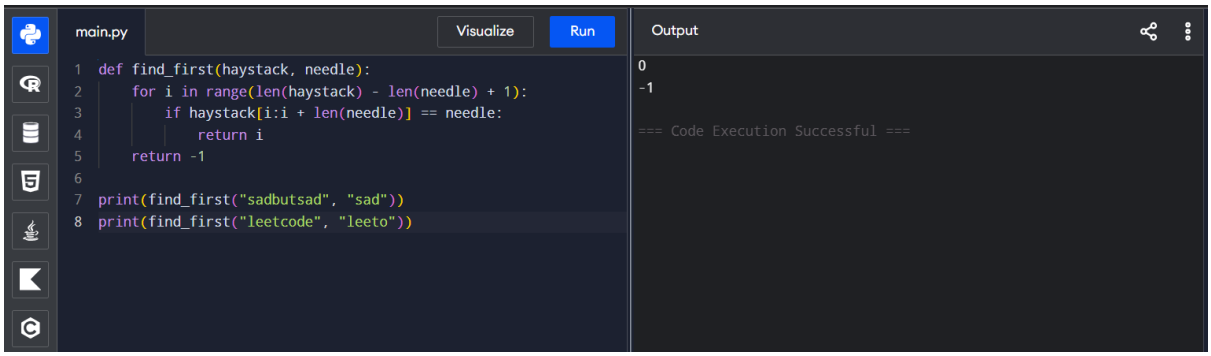
Output

```
9
6

=== Code Execution Successful ===
```

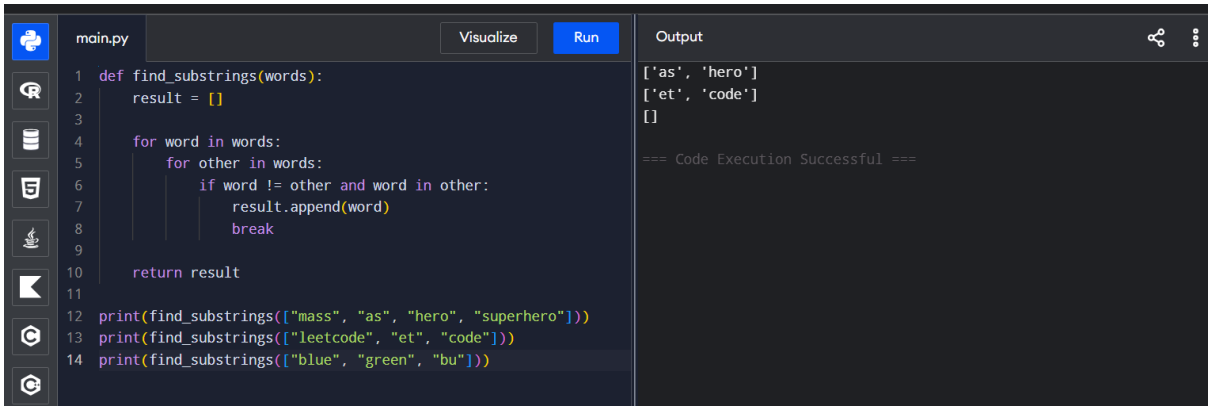
EXPERIMENT – 6



EXPERIMENT – 7



EXPERIMENT – 8



EXPERIMENT – 9

main.py

```python
import math
def closest_pair(points):
    min_dist = float('inf')
    pair = ()
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = math.sqrt((points[i][0] - points[j][0])**2 +
                          (points[i][1] - points[j][1])**2)
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])
    return pair, min_dist
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, distance = closest_pair(points)
print("Closest pair:", pair[0], "-", pair[1])
print("Minimum distance:", distance)
```

Output

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979

=== Code Execution Successful ===
```

EXPERIMENT – 10

main.py

```python
import math
def distance(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)
def closest_pair(p):
    best = float('inf')
    pair = None
    for i in range(len(p)):
        for j in range(i+1, len(p)):
            d = distance(p[i], p[j])
            if d < best:
                best, pair = d, (p[i], p[j])
    return pair, best
def convex_hull(p):
    hull = []
    for i in range(len(p)):
        for j in range(i+1, len(p)):
            side = []
            for k in range(len(p)):
                if k != i and k != j:
                    x = (p[j][0]-p[i][0])*(p[k][1]-p[i][1]) -
                    \
                        (p[j][1]-p[i][1])*(p[k][0]-p[i][0])
```

Output

```
Closest Pair: ((9, 3.5), (7.5, 4.5))
Distance: 1.8027756377319946
Convex Hull: [(10, 0), (5, 3), (15, 3), (6, 6.5), (12.5, 7)]

=== Code Execution Successful ===
```

EXPERIMENT – 11

main.py

```python
def convex_hull(points):
    hull = []
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            side = []
            for k in range(len(points)):
                if k != i and k != j:
                    cross = ((points[j][0] - points[i][0]) *
                             (points[k][1] - points[i][1]) -
                             (points[j][1] - points[i][1]) *
                             (points[k][0] - points[i][0]))
                    side.append(cross)
            if all(x >= 0 for x in side) or all(x <= 0 for x
            in side):
                hull += [points[i], points[j]]
    return list(dict.fromkeys(hull))
points = [(1,1), (4,6), (8,1), (0,0), (3,3)]
print("Convex Hull:", convex_hull(points))
```

Output

```
Convex Hull: [(4, 6), (8, 1), (0, 0)]

=== Code Execution Successful ===
```

EXPERIMENT – 12

main.py

```python
def distance(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)
def tsp(cities):
    start = cities[0]
    best = float('inf')
    best_path = None
    for p in permutations(cities[1:]):
        path = [start] + list(p) + [start]
        total = 0
        for i in range(len(path)-1):
            total += distance(path[i], path[i+1])
        if total < best:
            best = total
            best_path = path
    return best, best_path
cities1 = [(1,2), (4,5), (7,1), (3,6)]
d1, p1 = tsp(cities1)
print("Test Case 1:")
print("Shortest Distance:", d1)
print("Shortest Path:", p1)
cities2 = [(2,4), (8,1), (1,7), (6,3), (5,9)]
d2, p2 = tsp(cities2)
```

Output

```
Test Case 1:
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Test Case 2:
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]

=== Code Execution Successful ===
```

EXPERIMENT – 13

main.py

```python
from itertools import permutations
def total_cost(a, c):
    return sum(c[i][a[i]] for i in range(len(a)))
def assignment_problem(c):
    best = float('inf')
    best_a = None
    for a in permutations(range(len(c))):
        x = total_cost(a, c)
        if x < best:
            best, best_a = x, a
    return best_a, best
tests = [
    [[3,10,7],[8,5,12],[4,6,9]],
    [[15,9,4],[8,7,18],[6,12,11]]
]
for i, c in enumerate(tests, 1):
    a, cost = assignment_problem(c)
    print("Test Case", i)
    print("Assignment:", a)
    print("Total Cost:", cost)
```

Output

```
Test Case 1
Assignment: (2, 1, 0)
Total Cost: 16
Test Case 2
Assignment: (2, 1, 0)
Total Cost: 17

=== Code Execution Successful ===
```

EXPERIMENT – 14

main.py

```python
from itertools import combinations
def total_value(items, values):
    return sum(values[i] for i in items)
def is_feasible(items, weights, capacity):
    return sum(weights[i] for i in items) <= capacity
def knapsack(weights, values, capacity):
    best = []
    best_value = 0
    for r in range(len(weights) + 1):
        for items in combinations(range(len(weights)), r):
            if is_feasible(items, weights, capacity):
                v = total_value(items, values)
                if v > best_value:
                    best_value = v
                    best = list(items)
    return best, best_value
tests = [
    ([2,3,1], [4,5,3], 4),
    ([1,2,3,4], [2,4,6,3], 6)
]
for i, (w,v,c) in enumerate(tests, 1):
    items, value = knapsack(w, v, c)
```

Output

```
Test Case 1
Optimal Selection: [1, 2]
Total Value: 8
Test Case 2
Optimal Selection: [0, 1, 2]
Total Value: 12

=== Code Execution Successful ===
```